SPRINTS

Multi-Agent AI Career & Assessment Orchestrator

Final Project Report

Project type	Multi-agent AI prototype
Core orchestration	LangGraph StateGraph
LLM provider	Google Gemini via LangChain
Data systems	SQLite and ChromaDB
Execution sandbox	Docker
Runtime	Python 3.10.8

Working prototype • Technical and implementation report

Executive Summary

The Sprints Multi-Agent AI Career & Assessment Orchestrator is a Python prototype that routes learner requests to specialized AI agents. The Career Advisor combines a SQLite learner profile with project-context retrieval from ChromaDB and structured Gemini generation. The Python Assessor executes submitted code inside a constrained Docker container and returns structured grading feedback. A LangGraph workflow coordinates routing, evaluation and optimization, and a conditional human review checkpoint for eligible placement recommendations.

Important scope boundary: this prototype does not connect to a live job-vacancy database, contact employers, or submit applications. Its knowledge collection contains project-context documents rather than the complete official Sprints curriculum.

Project Objectives

- Route learner requests to a career-advice or Python-assessment workflow.
- Ground career recommendations in learner records and retrieved project evidence.
- Execute learner Python code away from the host process using Docker restrictions.
- Evaluate and refine successful responses with a bounded evaluator-optimizer loop.
- Require human review for a defined subset of tier-1 placement recommendations.

---

1. Project Overview

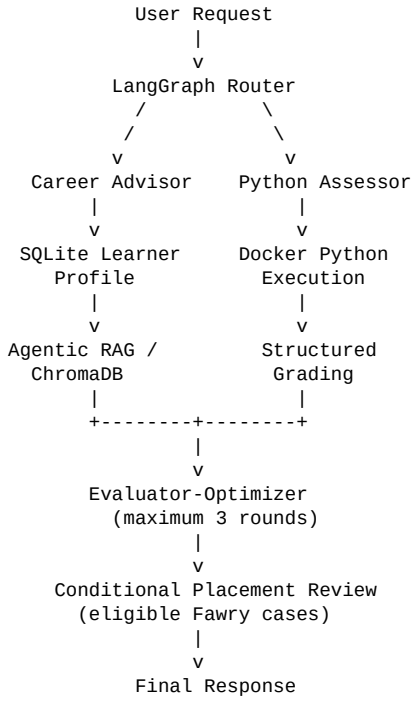
The Sprints Multi-Agent AI Career & Assessment Orchestrator is a Python prototype that routes learner requests to specialized agents. It currently supports two main workflows:

1. **Career Advisor:** retrieves a learner profile from SQLite, searches project knowledge in ChromaDB, evaluates retrieval quality, and generates structured career recommendations.
2. **Python Assessor:** executes submitted Python code in a restricted Docker container and evaluates the result against an assignment.

A central LangGraph workflow coordinates routing, agent execution, response evaluation/optimization, and a conditional human review step for selected placement recommendations.

The project is a prototype. It does not connect to a live jobs or vacancies database, does not contact employers, and does not submit job applications.

2. High-Level Architecture



The graph is implemented with `StateGraph(AgentState)`. A `MemorySaver` checkpointer is attached when the graph is compiled.

3. Technology Stack

The versions below are taken from the project's active virtual environment using `python -m pip freeze`. Python 3.10.8 is the interpreter version previously reported for this environment.

Component Version

--- ---

Python 3.10.8

LangChain 1.4.7

LangGraph 1.2.18

LangChain Core 1.7.3

LangChain Google GenAI 4.4.2

LangChain Chroma 1.1.0

LangChain HuggingFace 1.2.3

LangChain Text Splitters 1.1.3

ChromaDB 1.5.9

Pydantic 2.14.2

Sentence Transformers 6.2.0

Transformers 5.20.2

PyTorch 2.14.1

NumPy 2.2.6

Pandas 2.3.3

Google GenAI SDK 2.33.0

python-dotenv 1.2.4

python-pptx 1.0.2

pypdf 6.21.4

pytest 9.2.0

The complete environment snapshot is stored in the root `requirements.txt`. It contains all packages from the active environment, including transitive dependencies.

4. Repository Structure

```
sprints-ai-orchestrator/
■■■ data/
■   ■■■ curriculum_docs/
■   ■   ■■■ client_kickoff_deck.pptx
■   ■   ■■■ client_discovery_call.mp3
■   ■■■ processed/
■   ■   ■■■ client_kickoff_deck.md
■   ■   ■■■ client_discovery_call.md
■   ■   ■■■ project_brief.md
■   ■■■ sprints_learner_db.csv
■   ■■■ sprints_learners.sqlite3
■   ■■■ chroma_db/
■■■ sandbox/
■   ■■■ Dockerfile
■■■ src/
■   ■■■ __init__.py
■   ■■■ llm.py
■   ■■■ main.py
■   ■■■ state.py
■   ■■■ agents/
■   ■   ■■■ __init__.py
■   ■   ■■■ advisor_agent.py
■   ■   ■■■ assessor_agent.py
■   ■   ■■■ evaluator_optimizer.py
■   ■   ■■■ orchestrator.py
■   ■■■ tools/
■       ■■■ __init__.py
■       ■■■ audio_transcriber.py
■       ■■■ code_repl.py
■       ■■■ db_queries.py
■       ■■■ docker_executor.py
■       ■■■ document_loader.py
■       ■■■ rag_retriever.py
■■■ notebooks/
■■■ docs/
■■■ tests/
■   ■■■ __init__.py
■   ■■■ test_agents/
■       ■■■ test_agentic_rag.py
■       ■■■ test_evaluator_optimizer.py
■       ■■■ test_orchestrator_evaluator.py
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

This tree reflects the repository layout reviewed during documentation preparation. Generated `__pycache__` folders are omitted because they are Python runtime artifacts.

5. Shared State and Structured Models

`src/state.py` defines the shared `AgentState` TypedDict used by LangGraph. It includes fields for:

- Conversation messages and request routing.
- Learner ID and learner profile.
- Retrieved context, source metadata, and retrieval sufficiency.
- Submitted code, assignment, and expected output.
- Grading result and career advice.
- Evaluator feedback and iteration count.
- Human approval requirement, decision, and approval details.
- Errors and final response.

The state module also defines Pydantic models:

`GradingFeedback`

- `score`: integer from 0 to 100.
- `feedback_points`: one or more feedback items.
- `pass_status`: Boolean pass/fail result.

`CareerAdvice`

A structured career response containing the learner name, current track, skill assessment, recommended roles, skill gaps, learning roadmap, salary insight, and overall recommendation.

Using structured output helps keep agent responses aligned with the application's expected data shape.

6. Career Advisor Workflow

The Career Advisor implementation is in `src/agents/advisor_agent.py`.

6.1 Learner profile retrieval

The advisor first calls `get_learner_by_id()` to retrieve the learner record from SQLite. If the ID is missing or no matching record exists, the function returns an `Unable to verify`-style error rather than inventing a profile.

The database track field `bootcamp_track` is normalized into `current_track` for downstream use.

6.2 Agentic RAG

The advisor searches ChromaDB through `retrieve_context()` with a top-k value of 4. Each returned item retains its source and trust-level

metadata.

The retrieval process uses three possible attempts: 1. Initial retrieval. 2. First retry after query reformulation. 3. Second retry after another reformulation, if needed.

After each retrieval, a structured retrieval evaluator assesses relevance and evidence sufficiency. If the evidence is insufficient, a query reformulator creates a more focused query based on the missing information. The trace records the query, sources, document count, sufficiency, relevance score, reasoning, and missing information for each attempt.

The maximum is one initial attempt plus two retries (`MAX_RETRIEVAL_RETRIES = 2`).

6.3 Career recommendation generation

When retrieval is sufficient, the advisor passes the verified learner profile and retrieved context to a Gemini model configured for structured `CareerAdvice` output.

The prompt explicitly instructs the model to: - Use the SQLite learner profile as the source of learner facts. - Avoid inventing skills, qualifications, certifications, experience, or assessment results. - Identify skill gaps only when supported by evidence. - Avoid treating project requirements as official employer hiring requirements. - Avoid claiming that an employer has an active vacancy. - Distinguish verified facts from AI-generated recommendations.

The result includes the learner profile, structured advice, sources, retrieval trace, and evidence flags.

7. Python Assessor Workflow

The Python assessment implementation is in `src/agents/assessor_agent.py`.

The assessor receives: - Submitted Python code. - Assignment instructions. - Optional expected output.

The code is executed through `python_repl_tool`, which delegates execution to the Docker executor. If execution fails, the assessor returns an `Unable to verify` error result. If execution succeeds, the assessor uses Gemini structured output matching `GradingFeedback`.

The grading logic checks the submitted solution against the assignment

and execution information. The implementation uses a passing threshold of 60 in the assessor.

`src/main.py` currently demonstrates this workflow with two sample submissions: - A correct list sum and average implementation. - An incorrect average calculation.

The current `main.py` is an assessor demonstration entry point; it is not a complete interactive application interface for all agents.

8. Docker Code Execution

The Docker configuration is in `sandbox/Dockerfile`; execution controls are implemented in `src/tools/docker_executor.py`.

The sandbox image is based on `python:3.10-slim`, creates a non-root user with UID 10001, and runs from `/sandbox`.

The runtime configuration tested during development includes: - Network disabled. - Read-only container root filesystem. - Memory limit of 128 MB. - CPU quota of 0.5 CPU. - Process limit of 64. - All Linux capabilities dropped. - `no-new-privileges`. - A small temporary filesystem mounted with `noexec` and `nosuid`. - A five-second execution timeout. - Automatic container removal.

These controls reduce the risk of executing learner code directly on the host. They do not constitute a production-grade security boundary. In particular, the current executor captures process output without an explicit output-size cap. Do not expose this prototype to arbitrary public submissions without additional hardening and security review.

9. Evaluator-Optimizer

`src/agents/evaluator_optimizer.py` evaluates successful agent responses for quality and can request an improved response.

The workflow: 1. Evaluates the original response using a structured evaluation result. 2. If the response does not pass, generates an optimized response using evaluator feedback. 3. Repeats until the evaluator passes or the iteration limit is reached. 4. Preserves the original response if evaluation or optimization fails.

The orchestrator sets `MAX_OPTIMIZATION_ITERATIONS = 3`. The evaluator-optimizer tests use mocks, so they validate control flow without consuming Gemini API quota.

For assessment results, the orchestrator supplies the assignment,

expected output, submitted code, execution result, numerical score, and pass status as protected context. The evaluator prompt is instructed not to change execution facts or grading values.

10. LangGraph Orchestration and Human Review

`src/agents/orchestrator.py` defines and compiles the main graph.

Routing

The router uses the explicit `request_type` first. Otherwise, submitted code plus an assignment routes to the assessor; a learner ID routes to the advisor. Unrecognized requests go to the error node.

Evaluation routing

Successful advisor and assessor results are sent to the evaluator-optimizer. Failed or unverifiable results bypass optimization and end the workflow.

Human-in-the-Loop placement review

The current prototype treats Fawry as a tier-1 employer. A human review is requested only when: - Career advice succeeded. - The verified learner profile lists Fawry as a preferred employer. - The learner's readiness status is `Ready for Placement`.

The placement review node calls LangGraph `interrupt()` with a review payload. The graph can then be resumed with a human decision using `Command(resume=...)` and the same thread/checkpoint context.

The decision records approval, reviewer, comments, learner ID, and employer review. Approval applies only to the recommendation. It does not verify a live vacancy, confirm employer acceptance, contact the employer, or submit an application.

Checkpointing limitation

The graph is compiled with `MemorySaver`. This is in-memory checkpointing for the running process; it should not be described as durable storage across application restarts.

11. Learner Database and Tools

`src/tools/db_queries.py` provides read-only SQLite access. The SQL query tool is restricted to SELECT queries and opens the database in read-only mode. Learner lookup uses a parameterized query.

The database currently contains sample learner records. For example, the

reviewed `SPR-1001` record contains a learner track, completed module count, Python assignment score, target salary, preferred employers, and readiness status. This is sample project data, not a live Sprints learner system.

Other tool modules include: - `document_loader.py`: document ingestion support. - `audio_transcriber.py`: audio transcription support. - `rag_retriever.py`: document chunking, embedding, ChromaDB ingestion, and retrieval. - `code_repl.py`: LangChain tool wrapper for Python execution. - `docker_executor.py`: Docker-backed execution.

12. Knowledge Sources and Evidence Limitations

The reviewed source materials include: - `client_kickoff_deck.pptx`. - `client_discovery_call.mp3` and its processed transcript. - `project_brief.md`.

Important provenance limitations: - Slides 3 and 4 of the kickoff deck were truncated in the available source. - The discovery-call transcript was generated by Gemini and has not been human-verified. - The current ChromaDB collection contains project-context documents, not the complete official Sprints curriculum. - The learner database does not contain a live job openings table.

Therefore, the system must not claim that it has verified current job vacancies, official employer hiring criteria, or complete curriculum coverage.

13. Installation and Configuration

Prerequisites

- Python 3.10.
- Docker Desktop with the Linux container engine available.
- A Google Gemini API key for live LLM calls.

Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Install dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Configure the API key

Create a `.env` file in the repository root:

```
GOOGLE_API_KEY=your_api_key_here
```

Do not commit `.env` or expose the API key in logs, screenshots, notebooks, or source control. The project loads the key through `python-dotenv`.

Build the sandbox image

Run from the repository root:

```
docker build -t sprints-python-sandbox:latest -f sandbox/Dockerfile .
```

Run the current assessor demonstration

```
python -m src.main
```

The current demonstration makes live Gemini calls during grading. If the API quota is exhausted, it may fail even when Docker execution works. Automated tests should use mocks and should not require a live API key.

14. Testing

The test suite currently contains three agent-focused test files:

- `tests/test_agents/test_agentic_rag.py`
- `tests/test_agents/test_evaluator_optimizer.py`
- `tests/test_agents/test_orchestrator_evaluator.py`

The user reported that the complete suite ran successfully with **9 tests passed** at the time of the project review.

The tests cover agentic retrieval behavior, evaluator-optimizer iteration limits and early stopping, orchestrator routing/evaluation behavior, and human placement review approval/rejection paths.

This reported result is a snapshot from the development session. Re-run the suite in the current environment to confirm the present state:

```
python -m pytest -v
```

There are no separate tool-focused test files in the reviewed `tests` directory. Tool-level coverage should be expanded, especially for database errors, retrieval failures, Docker timeouts, output limits, and malformed learner submissions.

15. Known Limitations and Recommended Improvements

Area Current limitation Recommended next improvement

Knowledge base Project context Ingest approved documents only; not curriculum material and

full official track document versions

curriculum

Source quality Discovery transcript is Human-review and

AI-generated and correct transcript

unverified

Kickoff deck Slides 3 and 4 are Obtain a complete

truncated source deck

Job matching No live vacancy Integrate a trusted

database jobs feed before

claiming vacancy

matches

Placement review Prototype Make employer tiers

Fawry-specific configurable and add

condition reviewer

authentication/audit

Checkpointing `MemorySaver` is Add a durable

in-memory checkpointer for

restart-safe sessions

Docker execution Captured output has no Enforce output, disk,

explicit size cap memory, CPU, process,

and wall-clock limits

Test coverage Current tests focus on Add isolated tests for

agents every tool and failure

mode

User interface `main.py` is a Build a validated API

demonstration script or UI layer

Observability Console/logging support Add structured traces,

is basic request IDs, and

privacy-aware logs

---

16. Conclusion

The current implementation demonstrates a multi-agent architecture with a LangGraph router, a retrieval-grounded career advisor, a Docker-backed Python assessor, an evaluator-optimizer loop, and a conditional human approval checkpoint.

The project has a working prototype structure and a reported passing

test suite. Its key constraints are the limited project-context knowledge base, lack of live vacancy data, in-memory checkpointing, and the need for further sandbox hardening and tool-level tests. These limitations should remain explicit in any demo, report, or presentation.